UNIVERSITÉ
PARIS DESCARTES
UFR DE MATHÉMATIQUES ET INFORMATIQUE

Plan de test

Calcul de relations spatiales directionnelles à partir de l'histogramme d'angle

UNIVERSITÉ PARIS DESCARTES
UFR DE MATHÉMATIQUES ET INFORMATIQUE

Plan de test

Calcul de relations spatiales directionnelles à partir de l'histogramme d'angles

Les informations d'identification du document		*Les éléments de vérification du document*	
Référence du document :	D7	**Validé par :**	M.Wendling
Version du document :	1.00	**Validé le :**	17/04/2023
Date du document :	8 avril 2023	**Soumis le :**	17/04/2023
Auteur(s) :	XIE Rolland Devasenaradjounayagar Damien Al Mohaize Rayan	**Type de diffusion :**	Document (.pdf)
		Confidentialité :	Standard / Étudiants UFR Maths-Info de l'université Paris Descartes

Mots clés : Frameworks; bibliothèque; environnement de test; environnement d'exécution; environnement de développement; JAVA ;histogramme; barycentre; centre de gravité; API; interface

Sommaire

Sommaire 3

1. Introduction (ou préambule) 5

1.1. Objectifs et méthodes 5

1.2. Cadre 5

1.3. Documents de référence 6

2. Guide de lecture 6

2.1. Maîtrise d'œuvre 6

2.2. Maîtrise d'ouvrage 7

3. Concepts de base 7

4. Tests fonctionnels 8

4.1. Pour chaque scenario : 8

4.1.1. Identification 8

4.1.2. Description 8

4.1.3. Contraintes 10

4.1.4. Dépendances 11

4.1.5. Procédure de test 12

5. Tests d'intégration 15

5.1. Pour chaque test d'intégration : 15

5.1.1. Identification 15

5.1.2. Description 16

5.1.3. Contraintes 21

5.1.4. Dépendances 21

5.1.5. Procédure de test 22

6. Tests unitaires 27

6.1. Pour chaque test unitaire : 27

6.1.1. Identification 27

6.1.2. Description 28

6.1.3. Contraintes 33

6.1.4. Dépendances 34

6.1.5. Procédure de test 34

7. Vérification de la documentation 38

8. Annexes 39

9. Glossaire 42

10. Références 43

11. Index 44

1. Introduction (ou préambule)

1.1. Objectifs et méthodes

Ce document a pour but de présenter les tests réalisés lors de l'élaboration du programme pour le projet L2P2, intitulé : **"Calcul de relations spatiales directionnelles à partir de l'histogramme d'angles"**.

Pour cela, ce document présentera dans un premier temps le contexte dans lequel s'inscrit le projet, son objectif ainsi qu'une présentation de l'équipe. Dans un second temps, une présentation des tests qui vise à garantir que chaque fonctionnalité de l'application fonctionne conformément aux exigences du logiciel, ainsi qu'une analyse du fonctionnement global du développement. Pour finir avec la présentation des tests unitaires. Finalement, ce document s'achèvera avec la vérification du document et des informations complémentaires.

Le développement du logiciel se fait en plusieurs parties. Commençant par une lecture d'image précise, permettant de distinguer 2 objets qui seront comparés afin d'obtenir leurs angles. Pour au final pouvoir les situer grâce à leur histogramme d'angle. Tout cela se fait dans le langage JAVA. Ainsi, pour le bon fonctionnement de ce développement et pour que le programme respecte bien les résultats attendus, on a fait des tests unitaires, d'intégration et des tests fonctionnels. Ces tests sont le sujet principal de ce document.

1.2. Cadre

Ce projet s'inscrit dans l'UE de projet de la licence d'informatique de l'UFR de Maths-Info de l'Université de Paris. L'objectif est de faire réaliser à 3 étudiants un vaste projet en quasi-autonomie et en essayant d'approcher une gestion la plus professionnelle possible du projet.

1.3. Documents de référence

De nombreux livrables ont été rendus tout au long du projet. Ces derniers ont tous été publiés sur la forge du projet et sont accessibles à tout moment. Certains participent grâce à leurs contenus, à l'élaboration de ce document. Ces documents sont :

- Cahier des charges
- Cahier de recette
- Dossier de conception

2. Guide de lecture

2.1. Maîtrise d'œuvre

La maîtrise d'œuvre rassemble l'ensemble des personnes travaillant à la réalisation du projet. Elle devra impérativement maîtriser les concepts de base du langage de programmation. Elle doit aussi pouvoir décrire les différents tests réalisés pour le programme (*cf partie 4,5 et 6*), en détaillant afin que le client puisse comprendre de manière claire et nette.

Pour ce projet, trois maîtres d'œuvre ont été désignés: Xie Rolland, Rayan Al Mohaize, Devasenaradjounayagar Damien.

Chacun des maîtres d'œuvre se verra être chef de projet durant 3 semaines consécutives avant de céder sa place à un autre. Le premier chef de projet sera Devasenaradjounayagar Damien, puis Rayan Al Mohaize, et enfin Xie Rolland.

Les maîtres d'œuvre sont encadrés et guidés par la maîtrise d'ouvrage. Elle est représentée par le responsable de projet : M. Wendling Laurent.

2.2. Maîtrise d'ouvrage

Le maître d'ouvrage est le client ayant commandité le projet. Les clients doivent pouvoir comprendre au mieux les tests effectués pour chaque fonctionnalité du programme. Pour cela, la lecture de ce document doit leur permettre de comprendre l'utilité et le rôle de chacun des tests utilisés en fonction des classes présentes pour l'élaboration du projet. (*Cf. partie 4,5 et 6)*.

La personne bénéficiant des résultats du projet est M. Wendling, qui est donc le maître d'ouvrage. M. Wendling sera également notre encadrant tout au long de la réalisation du projet. Le rôle de l'encadrant est de guider le groupe et de vérifier le bon déroulement du projet, tout en donnant en parallèle des conseils et directives. Une réunion hebdomadaire est prévue pour cela chaque vendredi à 19h00.

3. Concepts de base

Les concepts clés liés aux **Relations spatiales**, en rapport avec le plan de test, incluent les:

TESTS FONCTIONNELS[1] : **Les tests fonctionnels ont pour but de garantir que le fonctionnement constaté est identique à celui attendu dans des situations réelles d'utilisation**.

TESTS D'INTÉGRATION[2] : Dans le monde du développement **informatique**, le **test d'intégration** est une phase de **tests**, précédée par les **tests** unitaires et généralement suivis par les **tests** de validation, vérifiant le bon fonctionnement d'une partie précise d'un logiciel ou d'une portion d'un programme.

TESTS UNITAIRES[3] : Le test unitaire est un **processus de vérification d'une unique unité de logiciel, elle-même définie comme étant la plus petite partie non divisible d'un code source**.

4. Tests fonctionnels

Description des scénarios (ainsi que leur enchaînement) permettant de vérifier que l'application recouvre bien le périmètre fonctionnel qui a été défini lors de la phase de spécification. Le périmètre fonctionnel est défini dans le cahier des charges (expression fonctionnelle du besoin).

4.1. Pour chaque scénario:

4.1.1. Identification

1. public void testAfficherObjet()
2. void testAfficheAngles()
3. void testAfficherCentreGrav()
4. public void testAffichageBarycentre()
5. void testSauvegarde()

4.1.2. Description

1.public void testAfficherObjet()

-Il s'agit de vérifier si la méthode "afficherObjet()" de la classe "Objet" affiche correctement les résultats.

-Le caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testAfficherObjet()" est correct grâce à "assertEquals" ainsi que "assertNotNull".

2.void testAfficheAngles()

-Il permet de vérifier si la méthode “afficheAngles()” de la classe “Pixel” affiche correctement les angles à l’aide des méthodes “getAngles()” et “afficheAngles”.

-Les caractéristiques de l’environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe “Pixel” doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier l’affichage de la méthode avec un exemple donné.

3.void testAfficherCentreGrav()

-Il s’agit de vérifier si la méthode “afficherCentreGrav()” de la classe “Objet” affiche bien le centre de gravité.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testAfficherCentreGrav()” est correct grâce à “assertFalse”.

4.public void testAffichageBarycentre()

-Il s’agit de vérifier si la méthode “affichageBarycentre()” de la classe “Objet” retourne bien la valeur attendue pour un objet “Objet” et une variable qui sont données.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testAffichageBarycentre()” est correct grâce à “assertEquals”. Il fera une comparaison de 2 arguments.

5.void testSauvegarde()

-Il s'agit de vérifier si la méthode "sauvegarde()" de la classe "Objet" enregistre bien les données.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testSauvegarde()" est correct grâce à "assertFalse".

4.1.3. Contraintes

Les contraintes et dépendances pour le test fonctionnel sont :

- L'environnement de test doit disposer d'une installation fonctionnelle de toutes les dépendances nécessaires au code à tester, y compris des bibliothèques de traitement d'images et de fichiers.
- Le code à tester doit être disponible et compilé sur l'environnement de test.
- Les tests peuvent avoir certaines nécessités comme par exemple la création d'un dossier "Result" pour "void testSauvegarde()".
- Le test suppose que les méthodes sont correctement implémentées.

Pour référence, les éléments de documentation pertinents pourraient inclure :

Les spécifications de la classe Objet/Pixel, qui décrivent les méthodes et les attributs disponibles.

Les spécifications des méthodes qui font partie du test fonctionnel, qui décrivent son comportement attendu et ses entrées/sorties.

Les exigences du système ou des utilisateurs concernant les fonctionnalités des méthodes.

4.1.4. Dépendances

Avant de réaliser ce test fonctionnel, il peut être judicieux de mener les tests préalables suivants pour s'assurer de la bonne préparation de l'environnement et de la fiabilité du code à tester :

1. Test de compilation : s'assurer que le code source peut être compilé avec succès et qu'aucune erreur ne se produit lors de la compilation.

2. Test d'installation : vérifier que toutes les dépendances nécessaires sont installées et fonctionnent correctement sur l'environnement de test.

3. Test de création de l'objet : créer un objet de la classe Objet et vérifier que tous les attributs sont initialisés correctement et qu'aucune exception n'est levée.

4. Test d'ajout de couleurs : ajouter des couleurs à l'objet créé et vérifier que l'objet contient bien les couleurs ajoutées.

5. Test d'ajout de points : ajouter des points à l'objet créé et vérifier que l'objet contient bien les points ajoutés.

6. Test de définition du centre de gravité : définir le centre de gravité de l'objet créé et vérifier que le centre de gravité est correctement défini.

7. Test d'ajout d'un angle barycentre : ajouter un angle barycentre à l'objet créé et vérifier que l'angle barycentre est correctement ajouté.

Ces tests préalables peuvent aider à identifier et à résoudre les éventuelles erreurs ou problèmes liés à l'environnement ou au code avant de passer à l'exécution du test fonctionnel.

4.1.5. Procédure de test

Voici les procédures de test:

public void testAfficherObjet()

Données en entrée : la méthode crée un nouvel objet à tester, nommé objet. Des valeurs sont également définies pour la largeur (width) et la hauteur (height) de l'image.

Résultats attendus : la méthode createObjetImage() est appelée sur l'objet en utilisant les valeurs de width et height. Le résultat attendu est une BufferedImage non vide, de largeur width et de hauteur height.

Critères de validation : le test utilise la méthode assertNotNull() pour vérifier que la BufferedImage retournée par la méthode createObjetImage() n'est pas nulle. Ensuite, la méthode assertEquals() est utilisée pour vérifier que la largeur et la hauteur de la BufferedImage correspondent aux valeurs de width et height définies précédemment. Si la BufferedImage est nulle ou si les dimensions ne correspondent pas, le test échouera. Ce critère de validation permet de s'assurer que la méthode createObjetImage() génère une image non vide de la taille spécifiée.

void testAfficheAngles()

Les données en entrée pour ce test sont un objet "Objet" et un objet "Pixel". L'objet "Objet" contient des pixels tandis que l'objet "Pixel" est un point isolé.

Le résultat attendu est l'affichage des angles entre l'objet "Pixel" et les pixels de l'objet "Objet".

Le critère de validation est l'assertion "assertEquals", qui n'est pas utilisée dans ce cas, car la méthode testée n'a pas de valeur de retour.

void testAfficherCentreGrav()

Les données entrées sont un objet “`Objet`” contenant un pixel, sa couleur, ainsi que les dimensions d'une image.

Les résultats attendus :

La méthode "afficherCentreGravite()" calcule le centre de gravité de l'objet à partir de ses points et le positionne au centre de l'image spécifiée par les paramètres "width" et "height".

La méthode "createObjetImage()" crée une image de l'objet avec la même largeur et hauteur spécifiée.

La méthode "assertFalse(frame.isVisible())" vérifie que la fenêtre ne soit pas visible, c'est-à-dire qu'elle se ferme immédiatement après son ouverture.

Le critère de validation est la vérification que la fenêtre d'affichage de l'image se ferme immédiatement après son ouverture grâce à “assertFalse(..)”.

public void testAffichageBarycentre()

Données en entrée : la méthode crée un nouvel objet à tester, puis ajoute un angle de 2,5 à l'objet via la méthode addAngleBarycentre().

Résultats attendus : le test vérifie si la méthode toString() de l'objet affiche le message "-------\nLa methode du barycentre nous donne: 2.5 degré." Le résultat attendu est que la chaîne de caractères retournée par la méthode toString() de l'objet est égale à la chaîne de caractères définie dans la variable messageAttendu.

Critères de validation : le test utilise la méthode assertEquals() pour vérifier que la chaîne de caractères retournée par la méthode toString() de l'objet est égale à la chaîne de caractères définie dans la variable messageAttendu. Si les chaînes de caractères sont différentes, le test échouera. Ce critère de validation permet de s'assurer que la méthode toString() de l'objet retourne la chaîne de caractères attendue.

void testSauvegarde()

Description des données en entrée :

- L'objet est créé et des points sont ajoutés. Un angle barycentre et un centre de gravité sont également définis.

Résultats attendus :

- Un fichier de sauvegarde doit être créé avec un nom de fichier conforme à la convention de nommage.
- Le fichier doit contenir les informations sur la couleur, les points, l'angle barycentre et le centre de gravité de l'objet.

Critères de validation :

Vérification de la création du fichier de sauvegarde avec un nom de fichier correct.

Vérification du contenu du fichier créé pour les informations sur l'objet.

5. Tests d'intégration

Description des tests (ainsi que leur enchaînement) permettant de vérifier que les différents modules, paquetages … de l'application s'interfacent correctement. On distinguera les interfaces internes des interfaces externes à l'application.

5.1. Pour chaque test d'intégration :

5.1.1. Identification

1. public void testAddPoint()
2. public void testRetourX()
3. public void testRetourY()
4. public void testRetourneEnsemblePoints()
5. public void testTailleEnsemblePoints()
6. public void testRetourCentreGravite()
7. public void testRetourCentreGraviteX()
8. public void testRetourCentreGraviteY()
9. public void testCalculAngles()
10. void testCreationHistogramme()
11. void testCalculCentreGrav()
12. void testGetAngleBot()
13. void testGetAnglesObjet()
14. void testGetAnglesBarycentre()

5.1.2. Description

1.public void testAddPoint()

-Il s'agit de vérifier si la méthode "addPoint()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testAddPoint()" est correct grâce à 2 "assertEquals". Ils feront une comparaison de 2 arguments.

2.public void testRetourX()

-Il s'agit de vérifier si la méthode "retourX()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testRetourX()" est correct grâce à 2 "assertEquals". Ils feront une comparaison de 2 arguments.

3.public void testRetourY()

-Il s'agit de vérifier si la méthode "retourY()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testRetourY()" est correct grâce à 2 "assertEquals". Ils feront une comparaison de 2 arguments.

4.public void testRetourneEnsemblePoints()

-Il s'agit de vérifier si la méthode "retourneEnsemblePoints()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet", un objet "Pixel" ainsi qu'une liste chaînée qui sont donnés.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testRetourneEnsemblePoints()" est correct grâce à 2 "assertTrue".

5.public void testTailleEnsemblePoints()

-Il s'agit de vérifier si la méthode "tailleEnsemblePoints()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet", un objet "Pixel" et une variable qui sont donnés.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testTailleEnsemblePoints()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

6.public void testRetourCentreGravite()

-Il s'agit de vérifier si la méthode "retourCentreGravite()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testRetourCentreGravite()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

7.public void testRetourCentreGraviteX()

-Il s'agit de vérifier si la méthode "retourCentreGraviteX()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testRetourCentreGraviteX()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

8.public void testRetourCentreGraviteY()

-Il s'agit de vérifier si la méthode "retourCentreGraviteY()" de la classe "Objet" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testRetourCentreGraviteY()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

9.public void testCalculAngles()

-Il s’agit de vérifier si la méthode “CalculAngles()” de la classe “Objet” retourne bien la valeur attendue pour un objet “Objet” et une variable qui sont données.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si les angles ont été calculés grâce à “assertNotNull”.

10.void testCreationHistogramme()

-Il s’agit de vérifier si la méthode “creationHistogramme()” de la classe “Objet” permet de créer correctement un histogramme..

La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testCreationHistogramme()” est correct grâce à “assertEquals”, ainsi que “assertTrue” et “assertFalse”.

11.void testCalculCentreGrav()

-Il s’agit de vérifier si la méthode “calculCentreGrav()” de la classe “Objet” calcule bien le centre de gravité d’un objet.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testCalculCentreGrav()” est correct grâce à “assertEquals”.

12.void testGetAngleBot()

-Il s'agit de vérifier si la méthode "getAngleBot()" de la classe "Objet" retourne bien la valeur attendue..

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testGetAngleBot()" est correct grâce à "assertEquals".

13.void testGetAnglesObjet()

-Il permet de vérifier si la méthode "getAnglesObjet()" de la classe "Pixel" retourne bien la valeur attendue pour un objet "Objet" , un objet "Pixel" et une liste chaînée qui sont donnés.

-Les caractéristiques de l'environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe "Pixel" doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testGetAnglesObjet()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

14.void testGetAnglesBarycentre()

-Il permet de vérifier si la méthode "getAnglesBarycentre()" de la classe "Pixel" retourne bien la valeur attendue pour un objet "Objet" et un objet "Pixel" qui sont donnés.

-Les caractéristiques de l'environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe "Pixel" doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testGetAnglesBarycentre()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

5.1.3. Contraintes

Les contraintes liées à ces tests d'intégration peuvent inclure :

- Un environnement de test avec des bibliothèques ou des outils spécifiques requis pour le test.
- Une installation particulière de l'application ou du système pour permettre le test.
- Une intervention humaine spécifique pour exécuter les tests ou pour résoudre des problèmes rencontrés lors des tests.

Pour ce qui est de la documentation de référence pour le test d'intégration, elle peut inclure :

- Les spécifications de conception pour les différents modules et composants du système.
- Les spécifications de l'API et des interfaces entre les différents modules.
- Les rapports de test et les résultats de test précédents

5.1.4. Dépendances

[Lister et expliciter les tests à mener préalablement à la réalisation du test.]

Avant de réaliser ces tests d'intégrations, il faut avant tout pour le bon fonctionnement de ces tests, réaliser des tests préliminaires :

Configuration de l'environnement de test : Vérifier que l'environnement de test est correctement configuré pour exécuter le test. Cela inclut les dépendances, les variables d'environnement, les autorisations, etc.

Installation et configuration de l'application : S'assurer que l'application à tester est correctement installée et configurée avec les bonnes versions de bibliothèques et dépendances.

Tests unitaires préliminaires : Effectuer des tests unitaires préliminaires pour s'assurer que les composants individuels de l'application fonctionnent correctement.

Tests de performance : Effectuer des tests de performance pour s'assurer que l'application peut gérer la charge et les volumes de données attendus.

Tests des méthodes : Il s'agit ici de tester toutes les méthodes appelées par chaque fonction de test pour vérifier qu'elles fonctionnent correctement.

5.1.5. Procédure de test

public void testAddPoint()

La donnée qui est en entrée est un objet "Pixel" qui est créé avec les valeurs (10, 20,255).

Les résultats attendus sont la valeur de l'objet "Pixel" ainsi que la taille de l'ensemble des points qui est ici de 1.

Les critères de validations sont donc "assertEquals" qui vérifient que la taille de l'objet "Objet" est bien de 1, et que sa valeur correspond bien à la valeur de "Pixel".

public void testRetourX() | public void testRetourY()

Les données en entrée des tests sont deux objets "Pixel" qui sont créés avec les valeurs (10, 20, 255) et (30, 40, 255). Les deux objets "Pixel" sont ajoutés à l'objet "obj" en utilisant la méthode "addPoint(Pixel p)".

Les résultats attendus sont l'appel de la méthode "retourX/Y(int index)" qui doit renvoyer la valeur de la coordonnée x/y de l'objet "Pixel" à l'indice "index" dans la liste des points de l'objet "obj". La méthode "retourX/Y(int index)" est appelée avec les indices 0 et 1, ce qui signifie que les coordonnées x/y des deux objets "Pixel" ajoutés doivent être retournées respectivement.

Les critères de validation sont :

Les tests sont réussis si les méthodes "retourX(int index)" et "retourY(int index)" renvoient les valeurs de coordonnées x et y qui sont attendues pour chaque objet "Pixel".

La méthode "assertEquals()" est utilisée pour comparer les valeurs retournées par les méthodes avec les valeurs attendues. Si la comparaison est fausse, le test échoue et une erreur est affichée.

public void testRetourneEnsemblePoints()

Les données en entrée de cette méthode de test sont deux objets de type Pixel qui contiennent des informations sur leur position et leur couleur. Ces deux pixels sont ajoutés à un objet de type Objet via les méthodes addPoint(). La méthode à tester retourne un ArrayList de Pixel contenant tous les pixels ajoutés à l'objet.

Les résultats attendus de ce test sont que l'ArrayList retourné par la méthode retourneEnsemblePoints() contient exactement les deux objets de type Pixel qui ont été ajoutés précédemment.

Les critères de validation pour ce test sont que les deux assertions contenant assertTrue() doivent être valides pour que le test réussisse. Si le ArrayList ne contient pas l'un ou l'autre des pixels ajoutés à l'objet, le test échouera.

public void testTailleEnsemblePoints()

Les données en entrée de cette méthode de test sont les mêmes que celles de la méthode précédente. Cependant, cette fois, la méthode à tester retourne un entier qui correspond au nombre de pixels ajoutés à l'objet.

Les résultats attendus de ce test sont que l'entier retourné par la méthode tailleEnsemblePoints() correspond au nombre de pixels ajoutés à l'objet, qui est de 2 dans ce cas précis.

Le critère de validation pour ce test est que l'assertion contenant assertEquals() soit valide pour que le test réussisse. Si l'entier retourné par la méthode n'est pas égal à 2, le test échouera.

public void testRetourCentreGravite() | public void testRetourCentreGraviteX() | public void testRetourCentreGraviteY()

Les données en entrée de ces tests sont un objet “Objet” qui a été créé et initialisé avec un centre de gravité attendu, qui est un objet Pixel.

Les résultats attendus de ces tests sont que la méthode retourCentreGravite() retourne un objet Pixel dont le centre de gravité correspond au centre de gravité attendu dans le premier test. La coordonnée X correspond à la coordonnée X du centre de gravité attendu dans le deuxième test, et que la coordonnée Y correspond à la coordonnée Y du centre de gravité attendu dans le troisième test.

Les critères de validation pour ces tests sont que les deux assertions contenant assertEquals() soient valides pour que les tests réussissent. Si les valeurs ne correspondent pas aux valeurs attendues, alors les tests échoueront.

public void testCalculAngles()

Données en entrée : la méthode crée deux nouveaux objets à tester, nommés “objA” et “objB”. Des pixels sont ajoutés aux deux objets via la méthode retourneEnsemblePoints() et la méthode add() qui ajoute un nouveau pixel dans l'ensemble de points. Les pixels sont définis avec des coordonnées (x,y) et une valeur de 255.

Résultats attendus : la méthode calculAngles() est appelée sur “objA” avec “objB” en paramètre. Le résultat attendu est que les angles entre les pixels de “objA” et “objB” ont été calculés et enregistrés dans chaque objet.

Critères de validation : le test utilise une boucle for pour vérifier que chaque pixel dans “objA” à un angle non nul après l'appel de la méthode calculAngles(). La méthode assertNotNull() est utilisée pour vérifier que chaque pixel a un angle non nul. Si un pixel n'a pas d'angle, le test échouera.

void testCreationHistogramme()

Les données en entrée sont un objet “Objet” contenant une liste de points (Pixel) qui ont chacun une position (x,y) et une couleur. L'objet “Objet” est modifié par l'appel de la méthode creationHistogramme().

Les résultats attendus sont que la méthode créationHistogramme() calcule et stocke correctement un histogramme des angles de chaque point de l'objet. L'histogramme doit avoir une taille de 361 éléments (un élément pour chaque degré de 0 à 360). Le nombre d'angles dans l'histogramme doit être égal au nombre total d'angles des points de l'objet. Pour chaque angle présent dans les points de l'objet, l'histogramme doit contenir cet angle. Pour chaque angle absent, l'histogramme ne doit pas contenir cet angle.

Les critères de validation sont les assertions dans le test :

- La taille de l'histogramme doit être égale à 361.
- Le nombre total d'angles dans l'histogramme doit être égal au nombre total d'angles des points de l'objet.
- Pour chaque angle présent dans les points de l'objet, l'histogramme doit contenir cet angle.
- Pour chaque angle absent, l'histogramme ne doit pas contenir cet angle.

void testCalculCentreGrav()

Les données en entrée sont un objet “`Objet`” contenant trois `Pixel` de coordonnées respectives (1,2), (3,4) et (5,6).

Les résultats attendus sont les coordonnées du centre de gravité de l'objet, qui devrait être (3,4) étant donné que les trois points ont la même masse et sont symétriquement placés par rapport au point (3,4).

Les critères de validation sont donc la vérification des coordonnées du centre de gravité renvoyé par la méthode `retourCentreGravite()`, qui doit correspondre aux coordonnées attendues.

void testGetAngleBot()

Données en entrée :

Trois pixels de coordonnées (1, 1), (2, 2) et (3, 3), et de couleur blanche, qui sont ajoutés à l'objet "obj".

Un pixel de coordonnées (4, 4) et de couleur blanche, qui est ajouté à l'objet "otherObj".

Résultat attendu :

La méthode "calculAngles()" calcule les angles entre l'objet "obj" et l'objet "otherObj" en considérant le pixel de "otherObj" comme le "bas" de l'objet, puis crée un histogramme de ces angles.

La méthode "getAngleBot()" retourne la somme des angles dans l'intervalle [0, 180] de l'histogramme.

Critères de validation :

La méthode "assertEquals()" vérifie que le résultat obtenu par "getAngleBot()" est égal au résultat attendu. Si la somme des angles de l'histogramme est correcte dans l'intervalle [0; 180], le test réussit.

void testGetAnglesObjet()

Les données en entrée pour ce test sont un objet "Objet" et un objet "Pixel". L'objet "Objet" contient des pixels tandis que l'objet "Pixel" est un point isolé.

Le résultat attendu est une liste d'angles entre l'objet "Pixel" et l'objet "Objet".

Le critère de validation est l'assertion "assertEquals" qui compare la valeur attendue avec la valeur obtenue à partir de l'objet Pixel. Ici, on vérifie que la taille de la liste d'angles est égale à 2, ce qui signifie qu'il y a bien deux angles entre l'objet "Pixel" et l'objet "Objet".

void testGetAnglesBarycentre()

Les données en entrée pour ce test sont un objet "Objet" et un objet "Pixel". L'objet "Objet" contient des pixels tandis que l'objet "Pixel" est un point isolé.

Le résultat attendu est un angle entre l'objet "Pixel" et le barycentre de l'objet "Objet".

Le critère de validation est l'assertion "assertEquals" qui compare la valeur attendue avec la valeur obtenue à partir de l'objet Pixel. Ici, on vérifie que la valeur de l'angle est égale à 0.

6. Tests unitaires

Description des tests (ainsi que leur enchaînement) permettant de vérifier que le comportement de chaque fonction, méthode, etc. de l'application correspond de manière satisfaisante à ce qui a été défini lors de la phase de conception détaillée.

6.1. Pour chaque test unitaire :

6.1.1. Identification

1. public void testViolet()
2. public void testJaune()
3. public void testVert()
4. public void testAddColor()
5. public void testAddAngleBarycentre()
6. public void testRetourAngleBarycentre()
7. void testGetAngleTop()
8. void testGetAngleRight()
9. void testGetAngleLeft()
10. void testGetX()
11. void testGetY()
12. void testGetColor()
13. void testGetAnglesPixel()

6.1.2. Description

1.public void testViolet() :

-Le but du test est de vérifier si la constante “VIOLET” de la classe “Couleurs” renvoie bien la valeur entière -12320428.

-Les caractéristiques de l’environnement de test dépendent du contexte dans lequel est exécuté le test. Cela peut inclure des facteurs tels que l’environnement d’exécution, les bibliothèques ou les frameworks utilisés, ainsi que les données ou les ressources nécessaires pour exécuter le test.

-Le principe de réalisation du test est d’utiliser la méthode “assertEquals()” fournie par JUnit, afin de faire une comparaison entre la constante “VIOLET” et la valeur attendue -12320428. Si la valeur retournée par “VIOLET” correspond bien à la valeur attendue, alors le test est réussi, sinon le test échoue, indiquant qu’il y a une erreur dans le code.

2.public void testJaune()

-Le but du test est de vérifier si la constante “JAUNE” de la classe “Couleurs” renvoie bien la valeur entière -137436.

-Les caractéristiques de l’environnement de test dépendent du contexte dans lequel est exécuté le test. Cela peut inclure des facteurs tels que l’environnement d’exécution, les bibliothèques ou les frameworks utilisés, ainsi que les données ou les ressources nécessaires pour exécuter le test.

-Le principe de réalisation du test est d’utiliser la méthode “assertEquals()” fournie par JUnit, afin de faire une comparaison entre la constante “JAUNE” et la valeur attendue -137436. Si la valeur retournée par “JAUNE” correspond bien à la valeur attendue, alors le test est réussi, sinon le test échoue, indiquant qu’il y a une erreur dans le code.

3.public void testVert()

-Le but du test est de vérifier si la constante “VERT” de la classe “Couleurs” renvoie bien la valeur entière -14506108.

-Les caractéristiques de l’environnement de test dépendent du contexte dans lequel est exécuté le test. Cela peut inclure des facteurs tels que l’environnement d’exécution, les bibliothèques ou les frameworks utilisés, ainsi que les données ou les ressources nécessaires pour exécuter le test.

-Le principe de réalisation du test est d’utiliser la méthode “assertEquals()” fournie par JUnit, afin de faire une comparaison entre la constante “VERT” et la valeur attendue -14506108. Si la valeur retournée par “VERT” correspond bien à la valeur attendue, alors le test est réussi, sinon le test échoue, indiquant qu’il y a une erreur dans le code.

4.public void testAddColor()

-Il s’agit de vérifier si la méthode “addColor()” de la classe “Objet” retourne bien la valeur attendue pour un objet “Objet” donné.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testAddColor()” est correct grâce à “assertEquals”. Il fera une comparaison de 2 arguments.

5.public void testAddAngleBarycentre()

-Il s’agit de vérifier si la méthode “addAngleBarycentre()” de la classe “Objet” retourne bien la valeur attendue pour un objet “Objet” et une variable qui sont données.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testAddAngleBarycentre()” est correct grâce à “assertEquals”. Il fera une comparaison de 3 arguments.

6.public void testRetourAngleBarycentre()

-Il s’agit de vérifier si la méthode “retourAngleBarycentre()” de la classe “Objet” retourne bien la valeur attendue pour un objet “Objet” et une variable qui sont données.

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testRetourAngleBarycentre()” est correct grâce à “assertEquals”. Il fera une comparaison de 3 arguments.

7.void testGetAngleTop()

-Il s’agit de vérifier si la méthode “getAngleTop()” de la classe “Objet” retourne bien la valeur attendue..

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testGetAngleTop()” est correct grâce à “assertEquals”.

8.void testGetAngleRight()

-Il s’agit de vérifier si la méthode “getAngleRight()” de la classe “Objet” retourne bien la valeur attendue..

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testGetAngleRight()” est correct grâce à “assertEquals”.

9.void testGetAngleLeft()

-Il s'agit de vérifier si la méthode "getAngleLeft()" de la classe "Objet" retourne bien la valeur attendue..

-La caractéristique de l'environnement de test est effectuée dans un environnement de développement Java avec un framework de test tel que JUnit.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testGetAngleLeft()" est correct grâce à "assertEquals".

10.void testGetX()

-Il s'agit de vérifier si la méthode "getX()" de la classe "Pixel" retourne bien la valeur attendue pour un objet "Pixel" donné.

-Les caractéristiques de l'environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe "Pixel" doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testGetX()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

11.void testGetY()

-Il s'agit de vérifier si la méthode "getY()" de la classe "Pixel" retourne bien la valeur attendue pour un objet "Pixel" donné.

-Les caractéristiques de l'environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe "Pixel" doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test "testGetY()" est correct grâce à "assertEquals". Il fera une comparaison de 2 arguments.

12.void testGetColor()

-Il permet de vérifier si la méthode “getColor()” de la classe “Pixel” retourne bien la valeur attendue pour un objet “Pixel” donné.

-Les caractéristiques de l’environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe “Pixel” doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testGetColor()” est correct grâce à “assertEquals”. Il fera une comparaison de 2 arguments.

13.void testGetAnglesPixel()

-Il permet de vérifier si la méthode “getAnglesPixel()” de la classe “Pixel” retourne bien la valeur attendue pour un objet “Pixel” donné, ainsi qu’avec une liste chaînée donnée.

-Les caractéristiques de l’environnement de test peuvent inclure des facteurs tels que les frameworks utilisés ou bien le fait que la classe “Pixel” doit être disponible et correctement importée.

-Le principe de réalisation de ce test est de vérifier si le résultat attendu du test “testGetAnglesPixel()” est correct grâce à “assertEquals”. Il fera une comparaison de 2 arguments.

6.1.3. Contraintes

Les contraintes liées à ces tests pourraient inclurent :

1. Environnement de test particulier : Le test doit être exécuté dans un environnement de test approprié avec toutes les dépendances nécessaires pour exécuter le code de test.
2. Installation particulière : Le code de test peut nécessiter des installations particulières telles que des bibliothèques tierces ou des modules spécifiques.
3. Intervention humaine spécifique : Le test peut nécessiter une intervention humaine spécifique telle que l'approbation manuelle des résultats de test ou la configuration de certains paramètres de test.

Les documents faisant référence peuvent inclure :

-Dossier de conception : Explique les méthodes utilisées.

-Manuel d'installation : Explique l'installation du logiciel.

-Manuel d'utilisation : Permet de décrire comment utiliser le logiciel.

6.1.4. Dépendances

Avant de réaliser un test, il est important de s'assurer que toutes les conditions préalables sont remplies :

Configuration de l'environnement de test : Vérifier que l'environnement de test est correctement configuré pour exécuter le test. Cela inclut les dépendances, les variables d'environnement, les autorisations, etc.

Installation et configuration de l'application : S'assurer que l'application à tester est correctement installée et configurée avec les bonnes versions de bibliothèques et dépendances.

Tests unitaires préliminaires : Effectuer des tests unitaires préliminaires pour s'assurer que les composants individuels de l'application fonctionnent correctement.

Tests de performance : Effectuer des tests de performance pour s'assurer que l'application peut gérer la charge et les volumes de données attendus.

6.1.5. Procédure de test

public void testViolet() | public void testJaune() | public void testVert()

Les données en entrée pour ce test sont les constantes définies dans la classe "Couleurs", qui associent une valeur entière à chaque couleur.

Les résultats attendus sont les valeurs entières correspondantes pour chaque couleur, c'est-à-dire "-12320428" pour le violet, "-137436" pour le jaune et "-14506108" pour le vert.

Les critères de validation sont les assertions "assertEquals" qui comparent les valeurs attendues avec les valeurs obtenues à partir des constantes définies dans la classe "Couleurs". Si les valeurs correspondent, le test est considéré comme réussi, sinon, il échoue.

public void testAddColor()

Les données entrées sont : -La création d'un objet "Objet", dans lequel on ajoute une couleur.

Le résultat attendu est la valeur 255 en utilisant la méthode getColor().

Le critère de validation est une assertion "assertEquals" qui compare les valeurs en arguments.

public void testAddAngleBarycentre() | public void testRetourAngleBarycentre()

Données en entrée : la méthode crée un nouvel objet à tester, puis définit une valeur numérique de l'angleBarycentreAttendu à 2,5. Cet angle sera ajouté à l'objet à tester via la méthode addAngleBarycentre().

Résultats attendus : le test vérifie si la méthode addAngleBarycentre() a correctement ajouté l'angleBarycentreAttendu à l'objet. Le résultat attendu est que la valeur retournée par la méthode retourAngleBarycentre() de l'objet est égale à la valeur d'angleBarycentreAttendu définie au début du test pour le premier test, et 2.5 pour le deuxième test. La précision est définie à 0,001.

Critères de validation : le test utilise la méthode assertEquals() pour vérifier que la valeur retournée par la méthode retourAngleBarycentre() de l'objet est égale à la valeur d'angleBarycentreAttendu, avec une précision de 0,001. Si la valeur retournée est différente, le test échouera. Ce critère de validation permet de s'assurer que la méthode addAngleBarycentre() fonctionne correctement et que l'objet retourne la valeur attendue.

void testGetAngleTop()

- Données en entrée :
    - Un objet "objet" vide est créé.
    - Un histogramme est créé dans l'objet "objet", avec des angles allant de 0 à 359 degrés. Les angles sont stockés dans une liste de listes de doubles.
- Résultat attendu :
    - La méthode "getAngleTop()" calcule la somme des angles de l'histogramme dont les valeurs sont dans l'intervalle [180; 359] degrés.
    - Le résultat obtenu est comparé avec un résultat attendu.
- Critères de validation :
    - La méthode "assertEquals()" vérifie que le résultat obtenu par "getAngleTop()" est égal au résultat attendu. Si la somme des angles de l'histogramme dans l'intervalle [180;359] est correcte, le test réussit.

void testGetAngleRight()

Données en entrée :

    - Un objet "objet" vide est créé.
    - Un histogramme est créé dans l'objet "objet", avec des angles allant de 0 à 359 degrés. Les angles sont stockés dans une liste de listes de doubles.
- Résultat attendu :
    - La méthode "getAngleRight()" calcule la somme des angles de l'histogramme dont les valeurs sont dans l'intervalle [90; 269] degrés.
    - Le résultat obtenu est comparé avec un résultat attendu.
- Critères de validation :
    - La méthode "assertEquals()" vérifie que le résultat obtenu par "getAngleRight()" est égal au résultat attendu. Si la somme des angles de l'histogramme dans l'intervalle [90; 269] est correcte, le test réussit.

void testGetAngleLeft()

Données en entrée :

- Un objet vide est créé.
- Pour chaque angle de 0 à 359, une liste de valeurs est ajoutée à l'histogramme de l'objet. Cette liste contient les valeurs de 0 à i-1.

Résultats attendus :

- La méthode getAngleLeft() calcule la somme des angles de l'histogramme dont les valeurs sont dans l'intervalle [0; 90] et [270;359] degrés.
- Le résultat obtenu est comparé avec un résultat attendu.

Critères de validation :

  - La méthode "assertEquals()" vérifie que le résultat obtenu par "getAngleLeft()" est égal au résultat attendu. Si la somme des angles de l'histogramme dans l'intervalle [0; 90] et [270;359] est correcte, le test réussit.

void testGetX() | void testGetY()

Les données en entrée pour ce test sont des objets de la classe "Pixel", qui ont chacun une coordonnée x et y et une couleur.

Les résultats attendus sont les coordonnées x et y pour chaque objet Pixel.

Les critères de validation sont les assertions "assertEquals" qui comparent les valeurs attendues avec les valeurs obtenues à partir de l'objet Pixel. Si les valeurs correspondent, le test est considéré comme réussi, sinon, il échoue.

void testGetColor()

Les données en entrée pour ce test sont des objets de la classe "Pixel", qui ont chacun une coordonnée x et y et une couleur.

Le résultat attendu est la valeur de la couleur pour chaque objet Pixel.

Le critère de validation est l'assertion "assertEquals" qui compare la valeur attendue avec la valeur obtenue à partir de l'objet Pixel. Si les valeurs correspondent, le test est considéré comme réussi, sinon, il échoue.

void testGetAnglesPixel()

Les données en entrée pour ce test sont des objets de la classe "Pixel", qui ont chacun une coordonnée x et y et une couleur.

Le résultat attendu est une liste d'angles pour chaque objet Pixel.

Le critère de validation est l'assertion "assertEquals" qui compare la valeur attendue avec la valeur obtenue à partir de l'objet Pixel. Ici, on vérifie que la taille de la liste d'angles est égale à 0, ce qui signifie que l'objet Pixel ne contient pas encore d'angles.

7. Vérification de la documentation

Tests unitaires :

Les tests unitaires seront effectués pour vérifier la conformité des différentes procédures décrites dans la documentation avec leur mise en œuvre réelle au niveau des unités de code. Les tests unitaires seront exécutés pour chaque fonction ou méthode du code, afin de s'assurer que chacune d'elles fonctionne correctement et produit le résultat attendu. Les tests unitaires seront exécutés à l'aide d'un framework de test automatisé.

Tests d'intégration :

Les tests d'intégration seront effectués pour vérifier la conformité de l'ensemble des procédures décrites dans la documentation avec leur mise en œuvre réelle au niveau de l'application. Les tests d'intégration seront exécutés pour chaque module de l'application, afin de s'assurer que les différentes fonctions ou méthodes sont correctement intégrées les unes aux autres et produisent le résultat attendu. Les tests d'intégration seront exécutés à l'aide d'un framework de test automatisé.

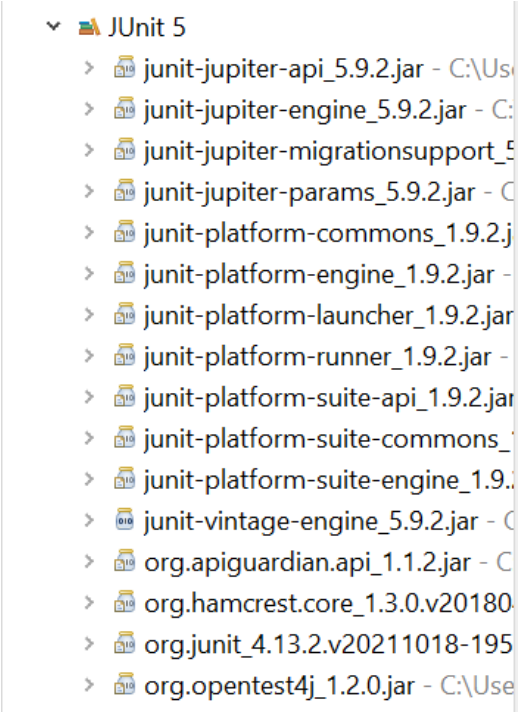
Tests fonctionnels :

Les tests fonctionnels seront effectués pour vérifier la conformité de l'ensemble des procédures décrites dans la documentation avec leur mise en œuvre réelle au niveau des exigences fonctionnelles de l'application. Les tests fonctionnels seront exécutés pour chaque scénario d'utilisation de l'application, afin de s'assurer que l'ensemble des fonctions ou méthodes sont correctement intégrées et répondent aux exigences fonctionnelles de l'application. Les tests fonctionnels seront exécutés manuellement à l'aide de cas de test.

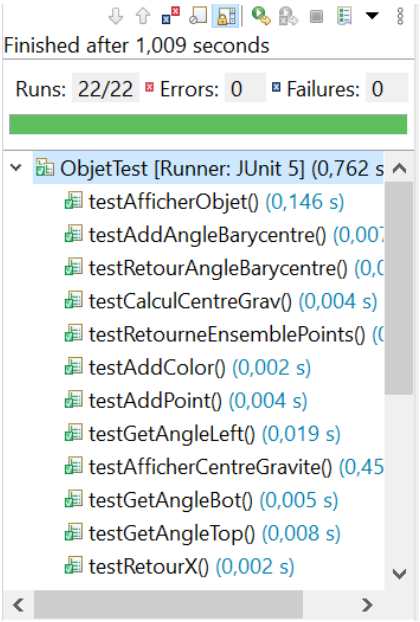
8. Annexes

JUnit :

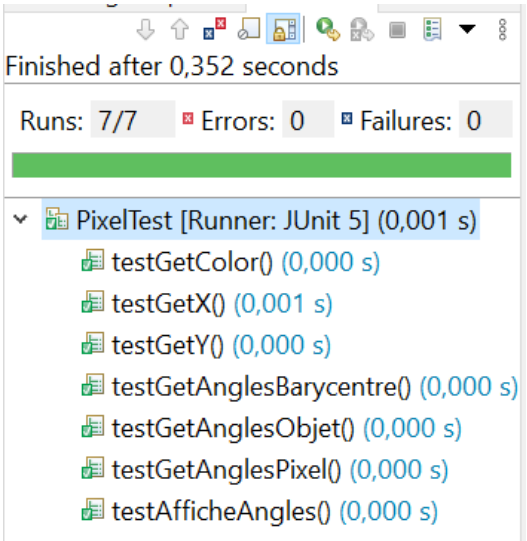
Notons d'ailleurs que si on veut utiliser JUnit, il faut impérativement créer un "JUnit test case" pour pouvoir lancer les tests unitaires.



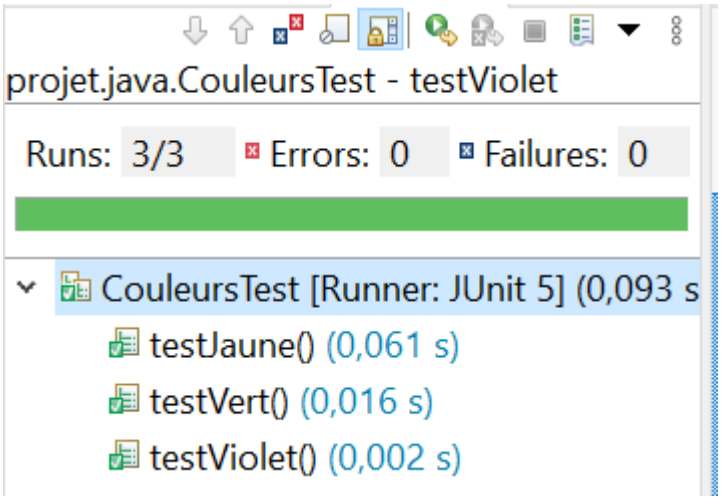
Résultat des tests :



ObjetTest.java



PixelTest.java



Scripts & Programme :

1. Pour les tests unitaires : JUnit pour Java, Mocha pour JavaScript, pytest pour Python, etc. Ces frameworks offrent des fonctions pour exécuter les tests unitaires, signaler les erreurs et mesurer la couverture de code.

2. Pour les tests d'intégration : Selenium pour les tests d'interface utilisateur, SoapUI pour les tests d'API, etc. Ces programmes permettent de simuler des conditions d'utilisation réelles pour tester le comportement du système dans son ensemble.

3. Pour les tests fonctionnels : Cucumber pour le comportement piloté par le développement (BDD),etc. Ces outils permettent de tester le comportement global du système en se concentrant sur les fonctionnalités et les résultats attendus.

9.Glossaire

-Framework : Un framework est un ensemble cohérent de composants logiciels structurels qui sert à créer les fondations ainsi que les grandes lignes de tout ou partie d'un logiciel, c'est-à-dire une architecture.

-Bibliothèque : En informatique, une bibliothèque logicielle est une collection de routines, qui peuvent être déjà compilées et prêtes à être utilisées par des programmes.

-Environnement de test : Un environnement de test est la configuration de types d'environnement pour un test particulier.

-Environnement d'exécution : Un environnement d'exécution ou runtime est un logiciel responsable de l'exécution des programmes informatiques écrits dans un langage de programmation donné.

-Environnement de développement : un environnement de développement est un ensemble d'outils qui permet d'augmenter la productivité des programmeurs qui développent des logiciels.

-JAVA : Langage de programmation orienté objet.

-Histogramme : Graphique qui montre la répartition des données en groupes. Il aide à visualiser la distribution des valeurs dans une population ou un échantillon.
On utilisera un histogramme pour représenter les résultats des angles obtenus via la méthode des paires de points.

-Barycentre : Le barycentre est le point où se trouve la masse moyenne d'un objet ou d'un système de masses. C'est le point de gravité de l'objet.

-Centre de gravité : le centre de gravité ou CdG, appelé G, est le point d'application de la résultante des forces de gravité ou de pesanteur.

-API : est une interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d'échanger des données et des fonctionnalités.

-Interface : Une interface définit la frontière de communication entre deux entités, comme des éléments de logiciel, des composants de matériel informatique, ou un utilisateur.

10. Références

Indique les références bibliographiques vers d'autres documents apportant des informations complémentaires

L'utilisation de classe ainsi que des packages ont été possibles grâce à la java documentation :

https://docs.oracle.com/javase/7/docs/api/

JUnit : https://www.cse.chalmers.se/edu/year/2012/course/TDA566/juniteclipse.html

Explication de JUnit : https://www.jmdoudoux.fr/java/dej/chap-junit.htm

Explication test intégration et fonctionnel :
https://openclassrooms.com/fr/courses/6100311-testez-votre-code-java-pour-realiser-des-applications-de-qualite/6616481-decouvrez-les-tests-dintegration-et-les-tests-fonctionnels

Java Exceptions (try…catch..) : https://www.w3schools.com/java/java_try_catch.asp
Mais on a également mieux compris grâce à nos cours de Programmation Orientée Objet.

11. Index

Liste les mots-clés du document et où les trouver dans celui-ci

- Framework - 4.1 ;5.1; 6.1; 9
- Bibliothèque - 4.1; 5.1; 6.1;9
- Environnement de test - 4.1; 5.1; 6.1; 9
- Environnement d'exécution - 6.1; 9
- Environnement de développement - 4.1; 5.1; 6.1; 9
- JAVA - 1.1; 4.1; 5.1; 6.1; 9
- Histogramme - 1.1; 5.1; 6.1; 9
- Barycentre - 4.1; 5.1; 6.1; 9
- Centre de gravité - 4.1; 5.1; 6.1; 9
- API - 5.1; 9; 10
- Interface - 5.1; 9